Edinburgh Napier UNIVERSITY

Our Interactive Visualisations So Far

Dr Pierre Le Bras

SET09123 – Interactive Data Visualisation

Lecture Summary

- A brief recap of D3 modules
  - Selection
  - Scale and Axis
  - Shape
  - Array
- Some JavaScript concepts too
  - Callbacks
  - Data Structures
- A summary of our application architecture
- Q&A

Edinburgh Napier UNIVERSITY

Data-Driven Documents

d3js.org

D3

- A JavaScript library for manipulating the DOM of web pages with data
- Free, open and framework independent
- Low-level declarative language
  - No built-in visualisation
  - Instructions to generate and modify the DOM, often with data

D3

Since version 4, it comes as a collection of modules:

- d3-array
- d3-axis
- d3-brush
- d3-chord
- d3-color
- d3-scale-chromatic
- d3-contour
- d3-delaunay
- d3-dispatch
- d3-drag
- d3-dsv
- d3-ease
- d3-fetch
- d3-force
- d3-format
- d3-geo
- d3-hierarchy
- d3-interpolate
- d3-path
- d3-polygon
- d3-quadtree
- d3-random
- d3-scale
- d3-selection
- d3-shape
- d3-time-format
- d3-time
- d3-timer
- d3-transition
- d3-zoom

D3

Edinburgh Napier UNIVERSITY

Manipulating the DOM

- d3-array
- **d3-axis**
- d3-brush
- d3-chord
- d3-color
- d3-scale-chromatic
- d3-contour
- d3-delaunay
- d3-dispatch
- d3-drag
- d3-dsv
- d3-ease
- d3-fetch
- d3-force
- d3-format
- d3-geo
- d3-hierarchy
- d3-interpolate
- **d3-path**
- d3-polygon
- d3-quadtree
- d3-random
- d3-scale
- **d3-selection**
- d3-shape
- d3-time-format
- d3-time
- d3-timer
- d3-transition
- d3-zoom

D3

Edinburgh Napier UNIVERSITY

Loading and operating data

- **d3-array**
- d3-axis
- d3-brush
- d3-chord
- **d3-color**
- **d3-scale-chromatic**
- d3-contour
- d3-delaunay
- d3-dispatch
- d3-drag
- **d3-dsv**
- d3-ease
- **d3-fetch**
- d3-force
- **d3-format**
- d3-geo
- **d3-hierarchy**
- d3-interpolate
- d3-path
- d3-polygon
- **d3-quadtree**
- **d3-random**
- **d3-scale**
- d3-selection
- d3-shape
- **d3-time-format**
- **d3-time**
- **d3-timer**
- d3-transition
- d3-zoom

D3

Edinburgh Napier UNIVERSITY

Creating visualisation components

- d3-array
- d3-axis
- d3-brush
- **d3-chord**
- d3-color
- d3-scale-chromatic
- **d3-contour**
- **d3-delaunay**
- d3-dispatch
- d3-drag
- d3-dsv
- d3-ease
- d3-fetch
- **d3-force**
- d3-format
- **d3-geo**
- d3-hierarchy
- d3-interpolate
- d3-path
- **d3-polygon**
- d3-quadtree
- d3-random
- d3-scale
- d3-selection
- **d3-shape**
- d3-time-format
- d3-time
- d3-timer
- d3-transition
- d3-zoom

D3

Edinburgh Napier UNIVERSITY

Implementing interaction and animation

- d3-array
- d3-axis
- **d3-brush**
- d3-chord
- d3-color
- d3-scale-chromatic
- d3-contour
- d3-delaunay
- **d3-dispatch**
- **d3-drag**
- d3-dsv
- **d3-ease**
- d3-fetch
- d3-force
- d3-format
- d3-geo
- d3-hierarchy
- **d3-interpolate**
- d3-path
- d3-polygon
- d3-quadtree
- d3-random
- d3-scale
- d3-selection
- d3-shape
- d3-time-format
- d3-time
- d3-timer
- **d3-transition**
- **d3-zoom**

D3

Edinburgh Napier UNIVERSITY

So far we have seen:

- **d3-array**
- **d3-axis**
- d3-brush
- d3-chord
- d3-color
- d3-scale-chromatic
- d3-contour
- d3-delaunay
- d3-dispatch
- d3-drag
- d3-dsv
- d3-ease
- **d3-fetch**
- d3-force
- d3-format
- d3-geo
- d3-hierarchy
- d3-interpolate
- d3-path
- d3-polygon
- d3-quadtree
- d3-random
- **d3-scale**
- **d3-selection**
- **d3-shape**
- d3-time-format
- d3-time
- d3-timer
- **d3-transition**
- d3-zoom

D3

Edinburgh Napier UNIVERSITY

We will also see these modules in the next weeks:

- d3-array
- d3-axis
- **d3-brush**
- d3-chord
- d3-color
- d3-scale-chromatic
- d3-contour
- d3-delaunay
- d3-dispatch
- d3-drag
- d3-dsv
- d3-ease
- d3-fetch
- d3-force
- **d3-format**
- **d3-geo**
- **d3-hierarchy**
- d3-interpolate
- d3-path
- d3-polygon
- d3-quadtree
- d3-random
- d3-scale
- d3-selection
- d3-shape
- **d3-time-format**
- d3-time
- d3-timer
- d3-transition
- **d3-zoom**

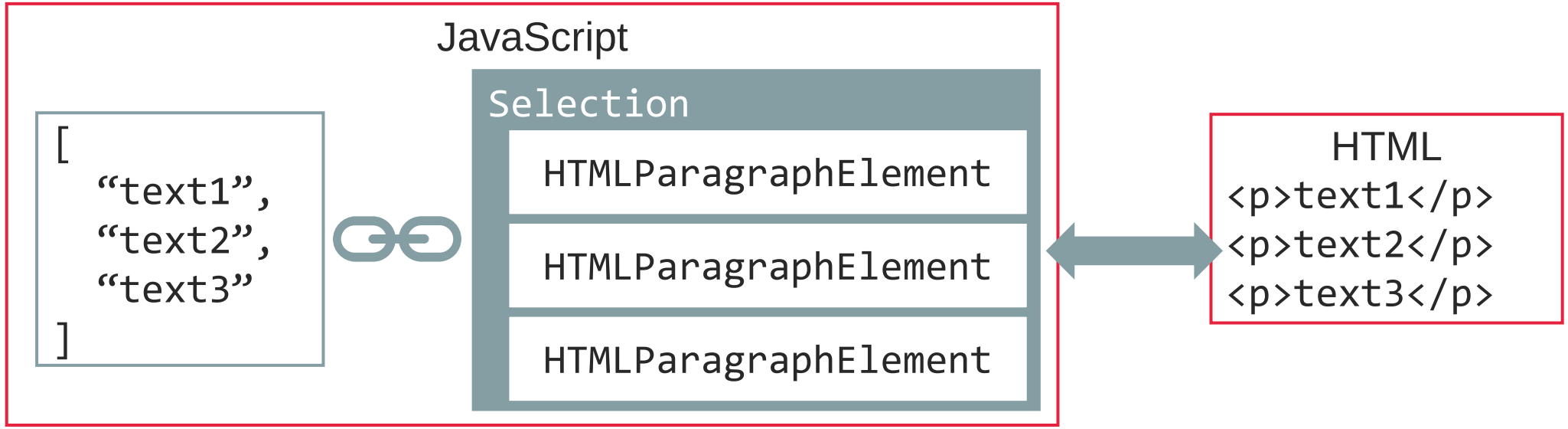
D3 Selection

github.com/d3/d3-selection

Selections

- Selections are wrapper object around JavaScript DOM objects
- They provide an API to create, modify and remove DOM elements with data

Selections


JavaScript
[
“text1”,
“text2”,
“text3”
]
Selection
HTMLParagraphElement
HTMLParagraphElement
HTMLParagraphElement
HTML
text1
text2
text3

Making Selections

- `d3.select(selector)` → get the first matching element
- `d3.selectAll(selector)` → get all matching elements
- `selection.selectAll(selector)` → get all matching child elements
- `selection.append(tag)` → create and return a child element

Binding Selections

- `selection.data(data, key)` → bind data to the selection
- `selection.join(tag)` → append/remove elements accordingly

Modifying Selections

- `selection.attr(name, value)` → modify an attribute
- `selection.style(name, value)` → modify a CSS style
- `selection.classed(class, bool)` → toggle a class

Adding Interaction

- `selection.on(event, callback)` → add an event handler
- `selection.transition([name])` → turn the selection into a transition

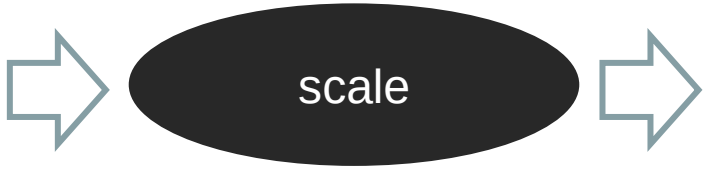
D3 Scale

github.com/d3/d3-scale

Scales

- Scales are functions mapping a data domain to a view range

```
data = [ 10 , 20 , 40, … ]
```




```
pixel = [ 1 , 5 , 10, … ]
```

Scales

- There are several scale available

		Range	
		Continuous	Discrete
Domain	Continuous	Linear, Log, Symlog, Power (Sqrt), Identity, Radial, Time (UTC) Using interpolators: Sequential, Diverging	Quantize, Quantile, Threshold
	Discrete	Band, Point	Ordinal

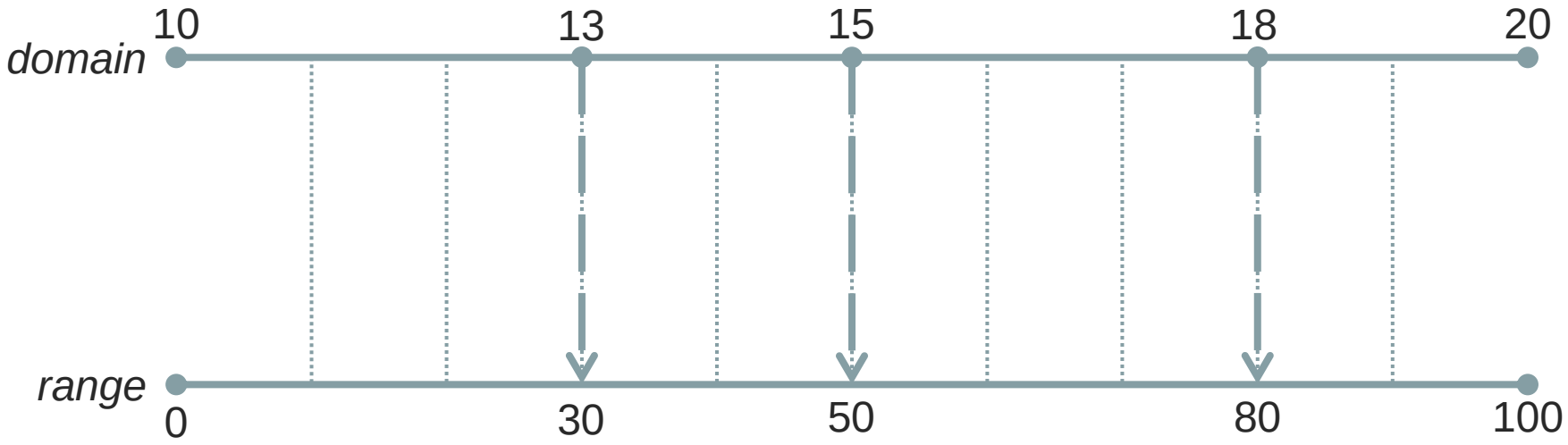
Implementing Scales

- `d3.scale([domain, range])` → create a new scale
- `scale.domain(domain)` → update the domain
- `scale.range(domain)` → update the range

- `scale(domainValue)` → get the matching range value

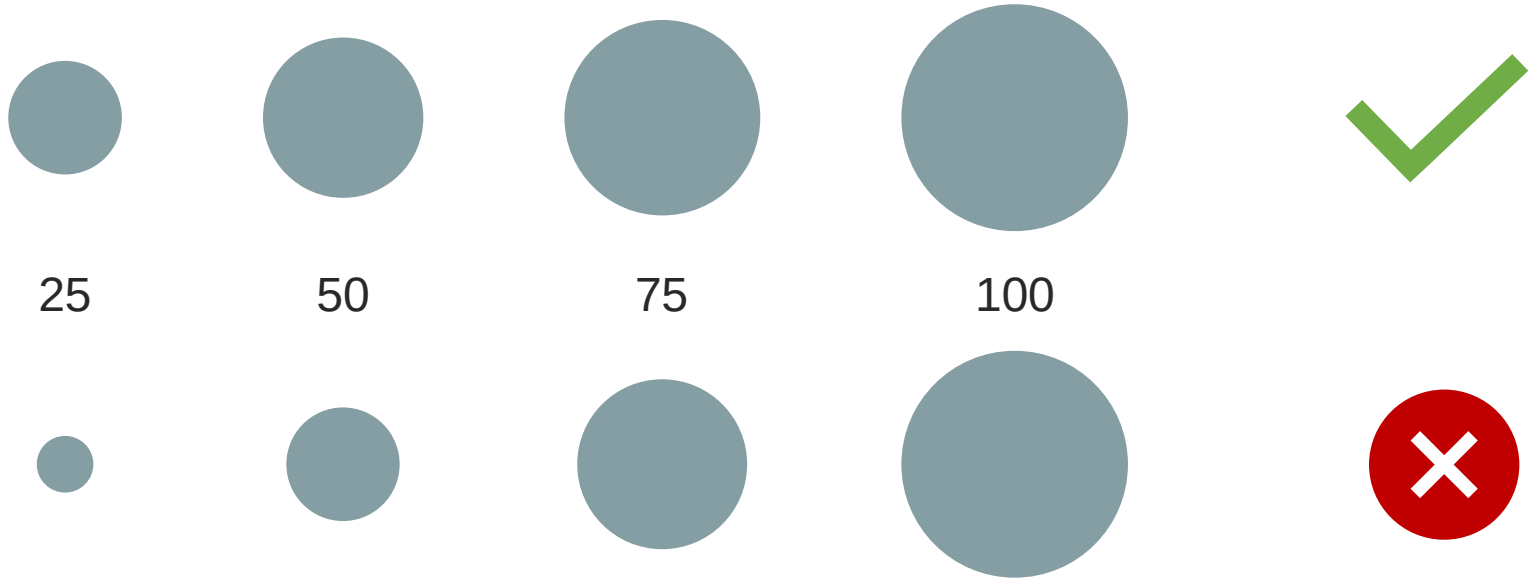
Linear Scale

```
let scale = d3.scaleLinear([10, 20], [0, 100]);
scale(13) // 30
```



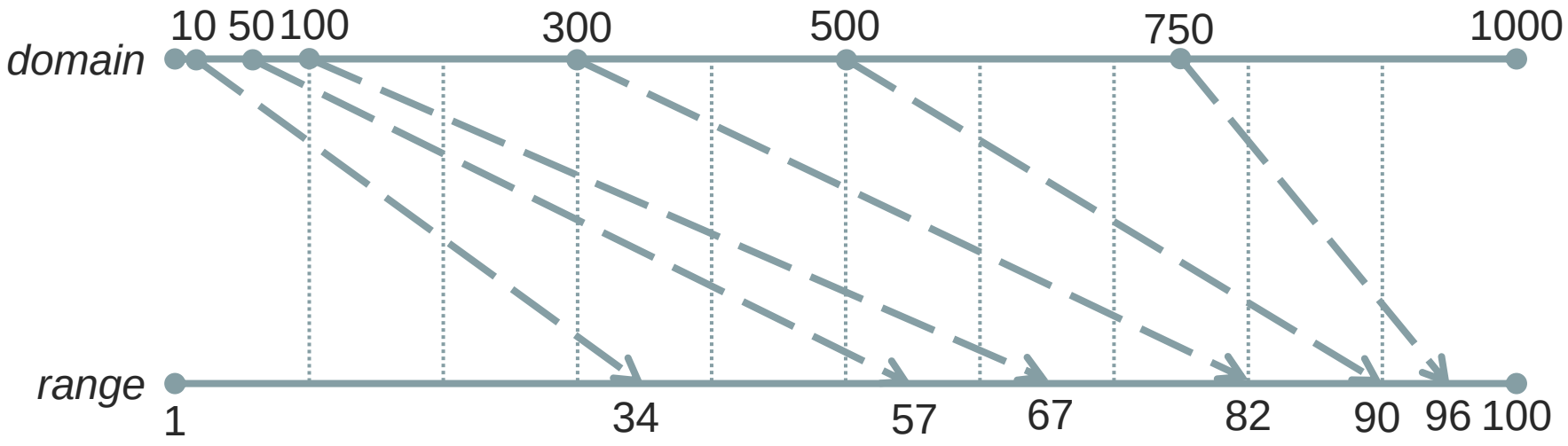
Square Root Scale

```
let scale = d3.scaleSqrt([0, 100], [0, 30]);
scale(25) // 15
scale(50) // 21.21
scale(75) // 25.98
```



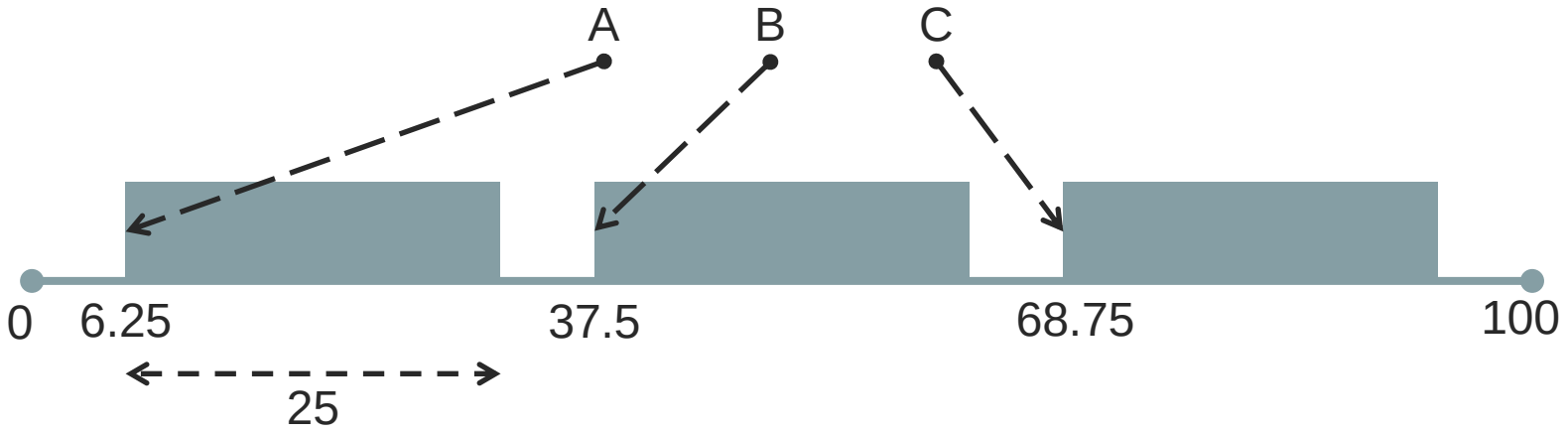
Log Scale

```
let scale = d3.scaleLog([1, 1000], [1, 100]);
scale(10) // 34
scale(100) // 67
scale(500) // 90
```



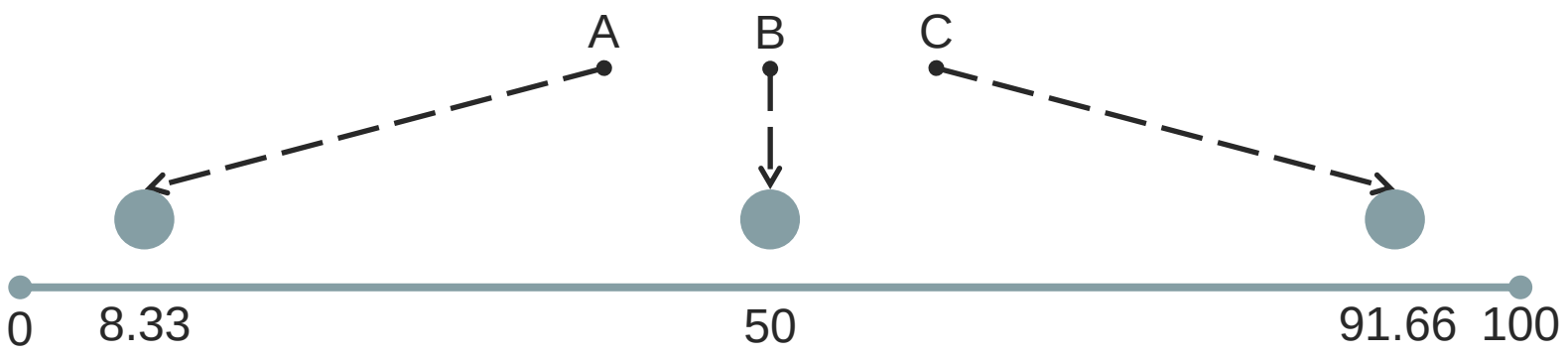
Band Scale

```
let scale = d3.scaleBand(['A','B','C'], [0,100]).padding(0.2);
scale('A') // 6.25
scale('B') // 37.5
scale('C') // 68.75
scale.bandwidth() // 25
```



Point Scale

```
let scale = d3.scalePoint(['A','B','C'], [0,100]).padding(0.2);
scale('A') // 8.33
scale('B') // 50
scale('C') // 91.66
```



D3 Axis

github.com/d3/d3-axis

Axes Generators

- Axes generator provide a quick way to display axes from scales

- `d3.axis(scale)` → create the axis generator
- `selection.call(axisGen)` → render the axis
- `axisGen.ticks(count, format)` → set tick format options

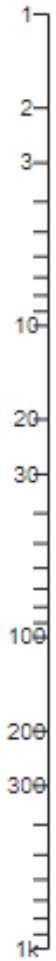
Axis Bottom

```
let s1 = d3.scaleLinear([0,500],[0,400]);

let axis = d3.axisBottom(s1);

mySelection.call(axis);
```

Axis Right

```
let s2 = d3.scalePoint(['A','B','C','D','F','E'],[0,400]);

let axis = d3.axisRight(s2);

mySelection.call(axis);
```

Axis Left




```
let s3 = d3.scaleLog([1,1000], [0,400]);

let axis = d3.axisLeft(s3);

mySelection.call(axis);
```

Axis Top

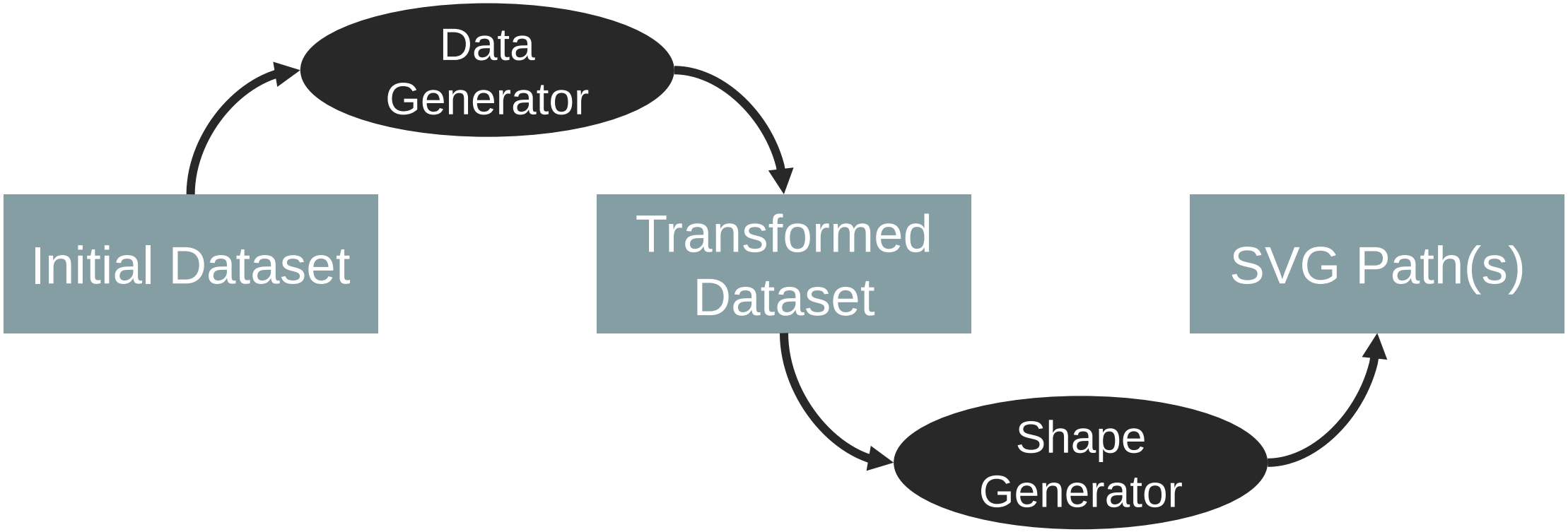
```
let s4 = d3.scaleTime([new Date(2016, 0, 1),
                       new Date(2022, 0, 1)], [0,400]);

let axis = d3.axisTop(s4);

mySelection.call(axis);
```

D3 Shape

github.com/d3/d3-shape

Generators

- The module provides data and shape generators
- Data generators transform data ready for visualisation
- Shape generators compute the path of complex shapes



Data Generators

- `d3.pie()` → create a data generator for pie and donut charts
- `d3.stack()` → create a data generator for stacked layouts

Shape Generators

- `d3.arc()` → render arcs for pie and donut charts
- `d3.line()` → render lines for line charts
- `d3.area()` → render areas for area charts
- `d3.link()` → render links between points, e.g., for trees
- `d3.symbol()` → render symbols for scatter plots

Curve Interpolators

- `d3.curveBasis` & `d3.curveBundle` → B-spline interpolation
- `d3.curveBumpX/Y` → pair-wise Bézier curves
- `d3.curveCardinal` → Cardinal spline interpolation
- `d3.curveNatural` → Cubic spline interpolation
- `d3.curveStep` → Horizontal and vertical line interpolation
- `d3.curveLinear` → pair-wise straight lines

D3 Array

github.com/d3/d3-array

JS Data Structures

Since ES6, JavaScript provides implementations for:

- Objects – Collection of values identified with a key
- Arrays – Collection of values identified with an index
- Maps – Collection of values identified with a key
- Sets – Collection of unique values

Array Operations

- `.push(value)` – adds a new element at the end of the array
- `.pop()` – removes the last element in the array
- `.sort(comparator)` – sorts the array given a comparator function
- `.every(test)` – checks if `test` holds true for all elements in the array
- `.some(test)` – checks if `test` holds true for at least one element
- `.forEach(callback)` – executes the callback, for each element in the array
- **`.map`**`(f)` – returns a new array, based on the transformation *f*
- **`.filter`**`(test)` – returns a new array with only elements for which *test* is true
- `.reduce(f)` – reduces the array to a single value, given the accumulator *f*
- `.join(s)` – creates a string concatenation of the array, joined by the given string

D3 Array

- The D3 Array module provides even more functions to explore and refine datasets

- Notably:
  - Getting descriptive statistics about the dataset
  - Transforming the dataset, notably by aggregation

Statistics

- Each method takes an iterable as first parameter, and an optional accessor function, e.g., to access specific attributes in object
  - Quantile needs a percentile parameter too

```
d3.min          d3.max          d3.extent
d3.mean         d3.mode         d3.median
d3.variance     d3.deviation    d3.quantile
d3.rank         d3.sum          d3.cumsum
```

Transformations – Group

- The group transformation aggregates the data by the provided key accessor
- If several keys are given, a nested aggregation is produced

- `d3.group` returns the aggregation as a (nested) Map
- `d3.groups` returns the aggregation as a (nested) Array
- `d3.flatGroup` returns the aggregation as a flat array

Transformations – Rollup

- The rollup transformation similarly aggregates the data by the provided key accessor, but also reduces the groups into a single value
- If several keys are given, a nested aggregation is produced

- `d3.rollup` returns the aggregation as a (nested) Map
- `d3.rollups` returns the aggregation as a (nested) Array
- `d3.flatRollup` returns the aggregation as a flat array

Transformations – Bins

- The bin generator distributes the data into non-overlapping, consecutive intervals
- To instantiate a bin generator:

```
d3.bin()
```

- `bin.values` lets you set the value accessor
- `bin.domain` lets you set the domain accessor
- `bin.thresholds` lets you set the number of bins to create, by number or formula

Fetching Data

- D3's fetch module provides functionalities to retrieve and parse data from several formats (json, dsv, xml, etc.)

- It uses asynchronous callbacks, or *Promises*

```
d3.csv('myDataset.csv')
    .then(data=>{
        // when the fetch is complete
    })
    .catch(error=>{
        // if an error occur during fetch
    })
    .finally(()=>{
        // whatever happens execute this
    })
```

Edinburgh Napier UNIVERSITY

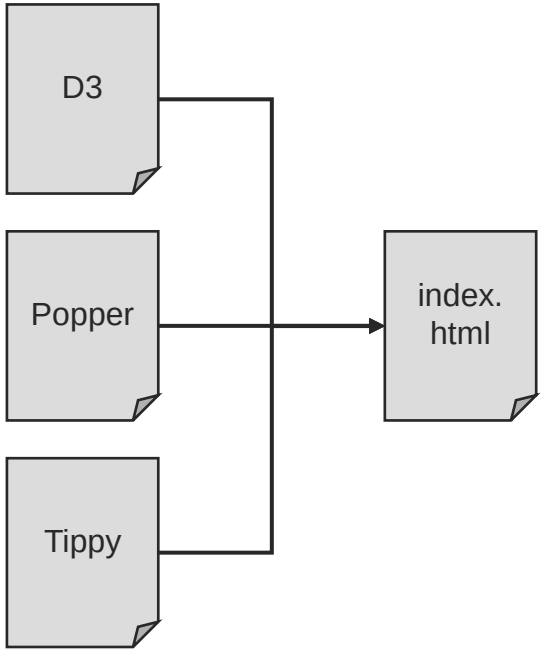
Application Architecture

Our App Architecture

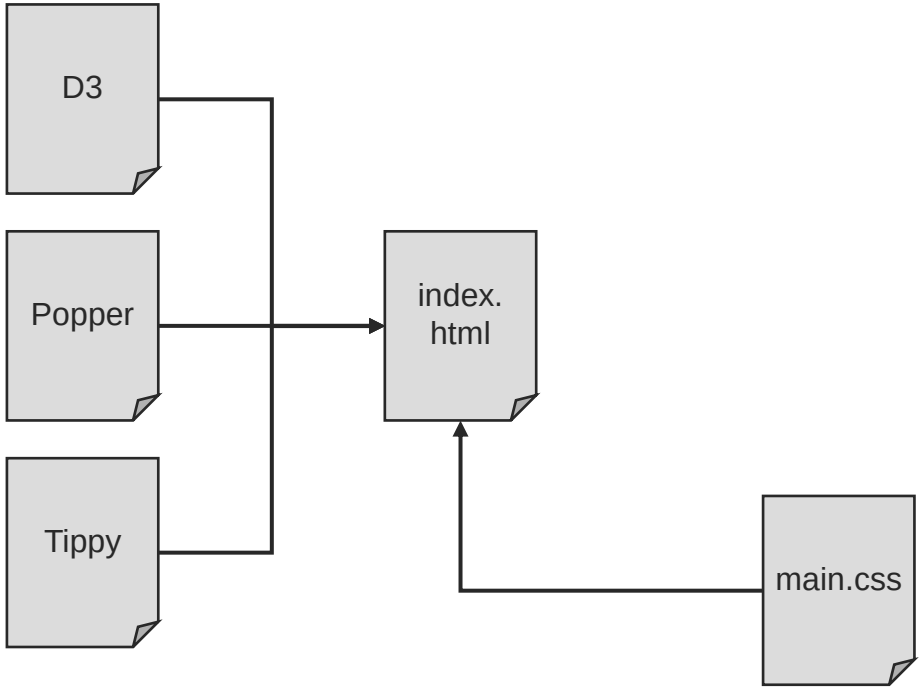
Edinburgh Napier UNIVERSITY

Our App Architecture

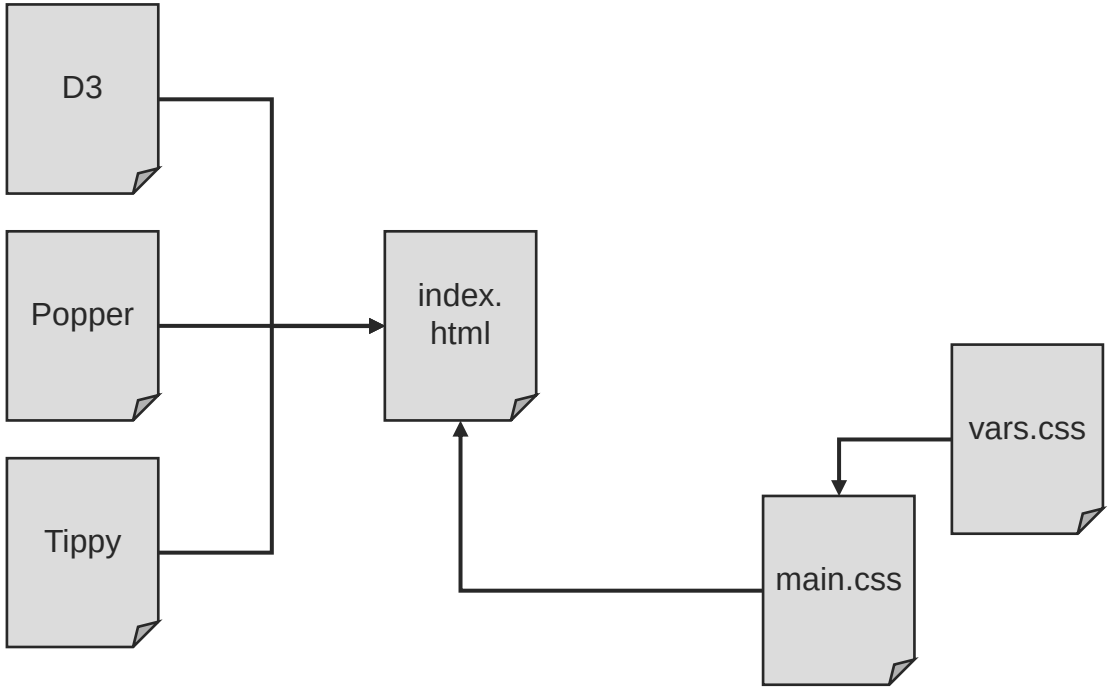
Our App Architecture



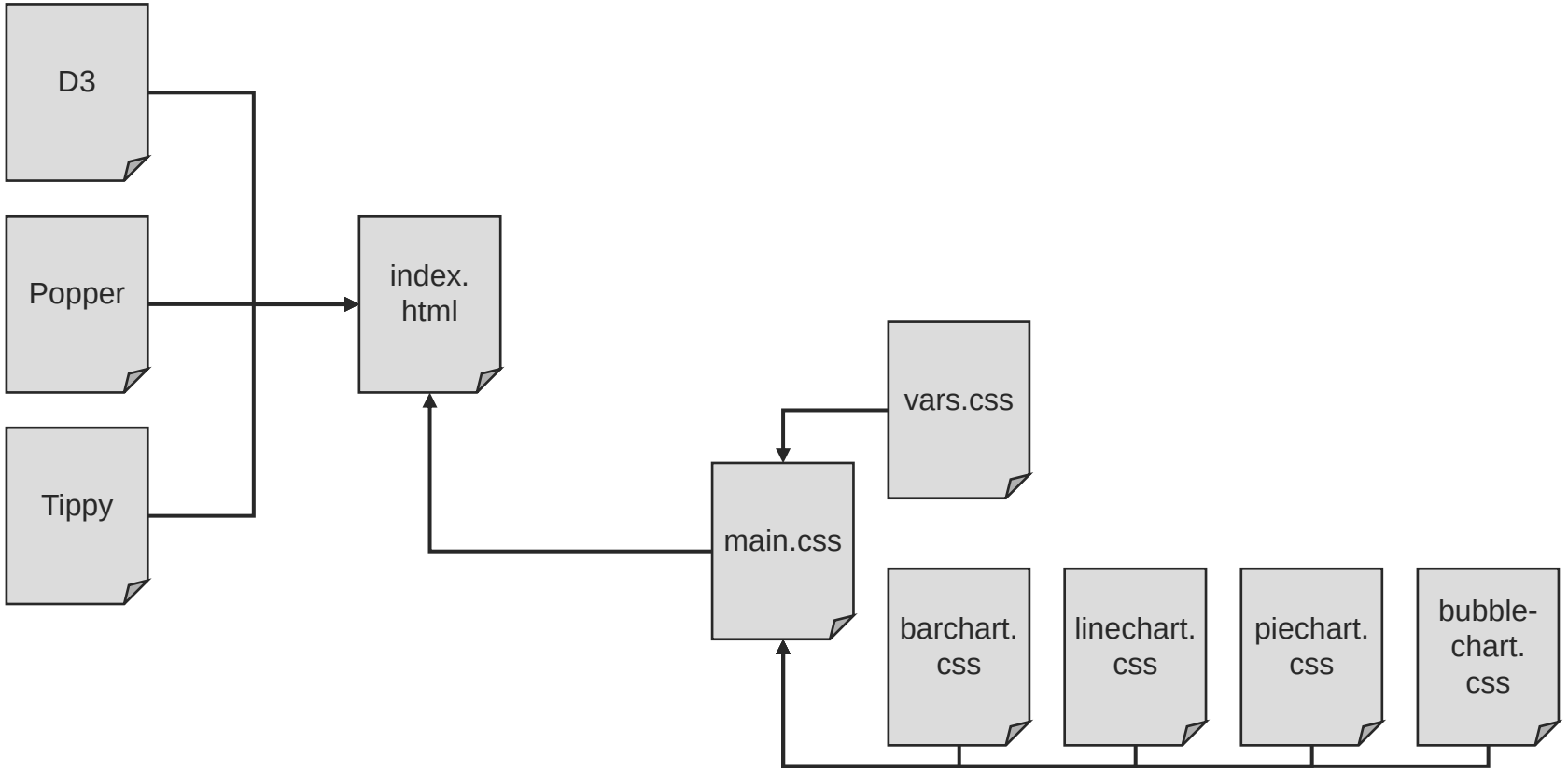
Our App Architecture



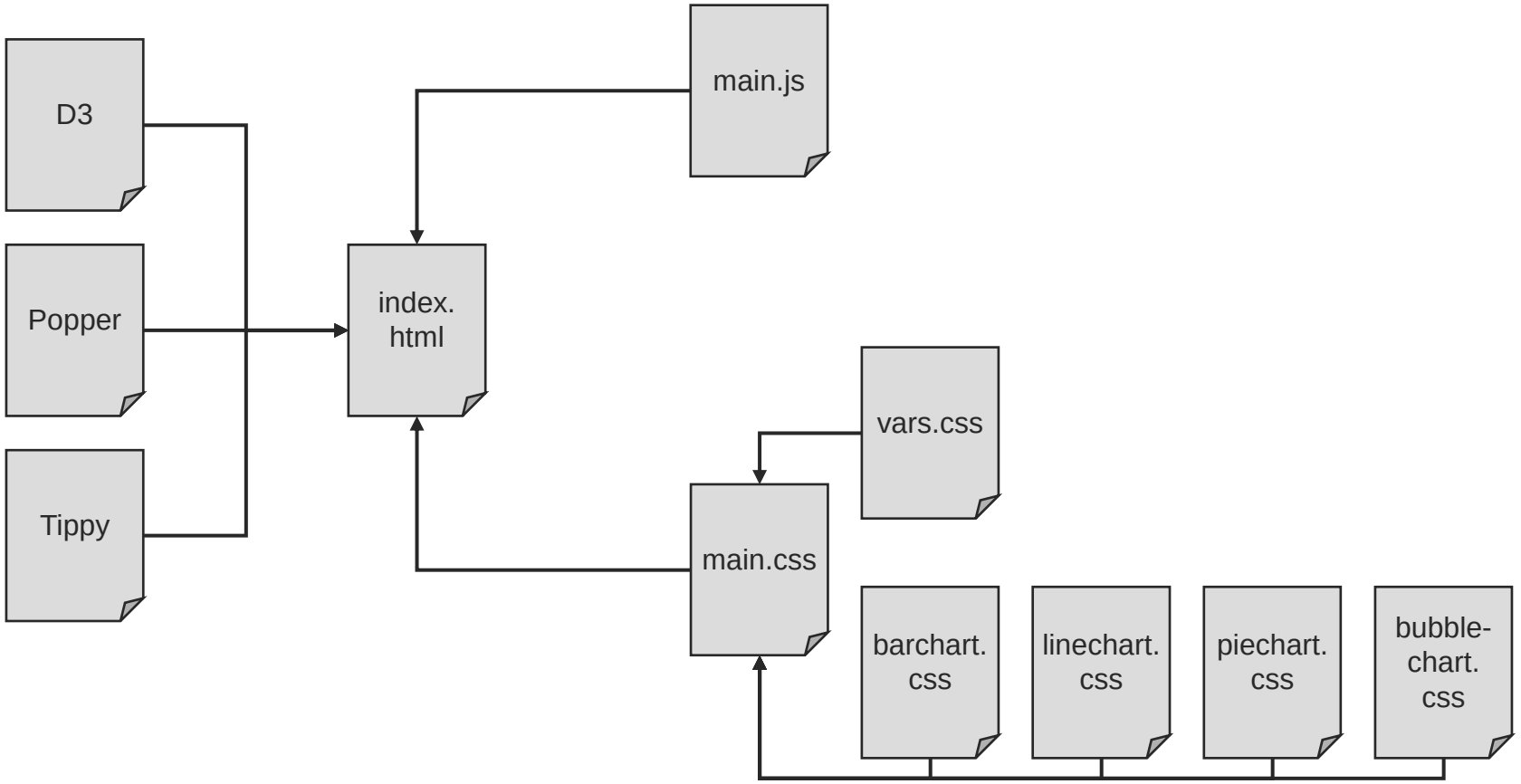
Our App Architecture



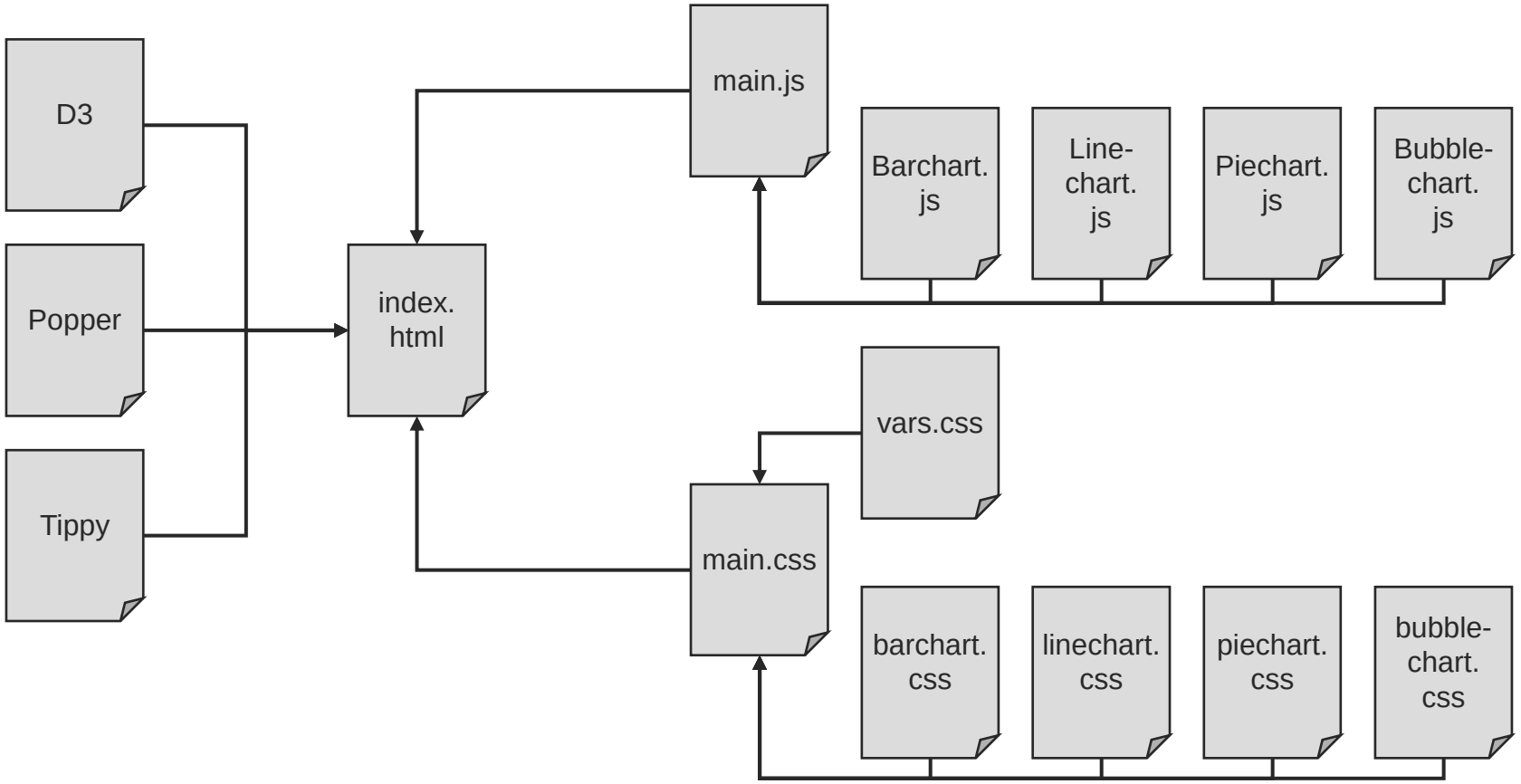
Our App Architecture



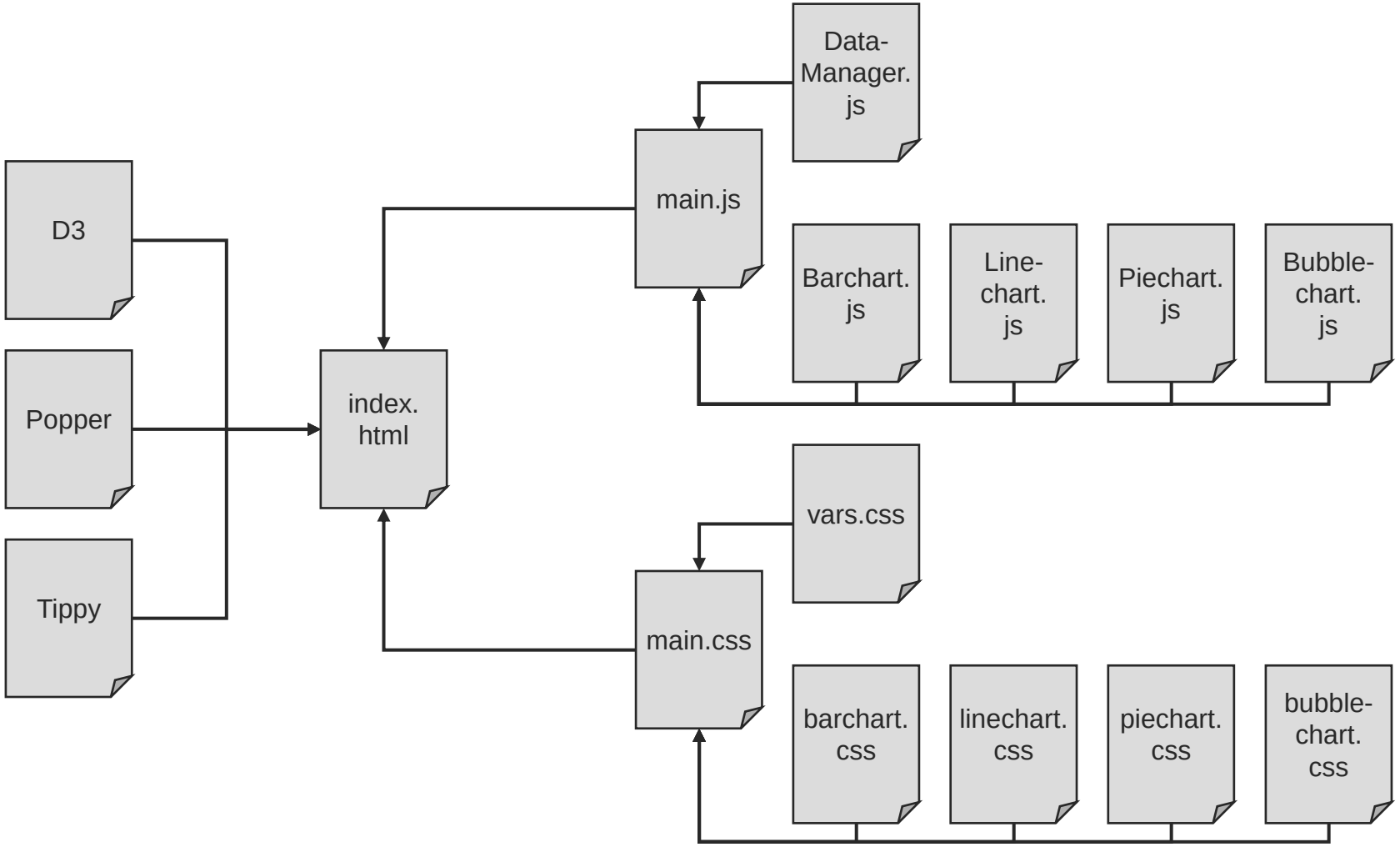
Our App Architecture



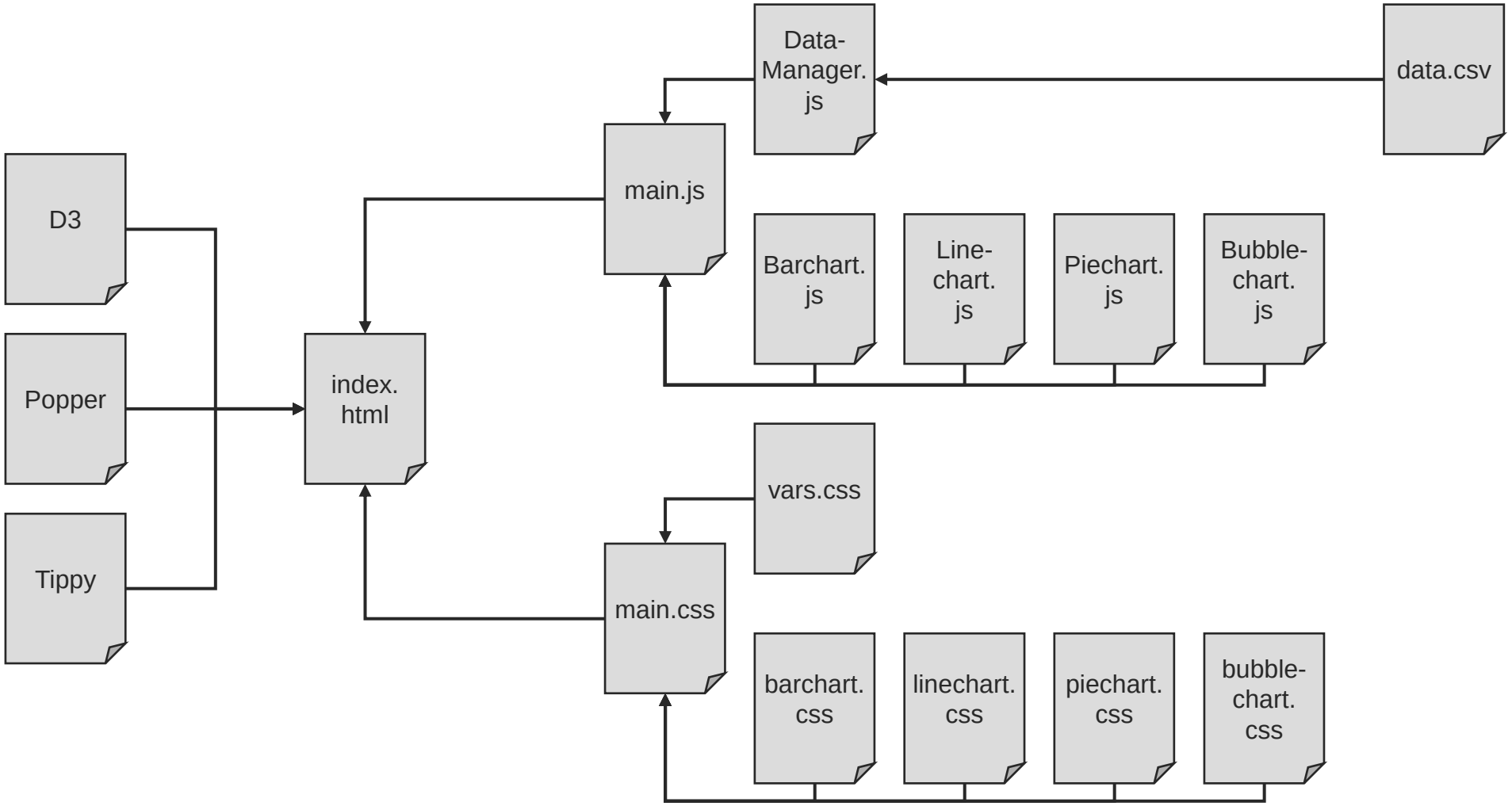
Our App Architecture



Our App Architecture

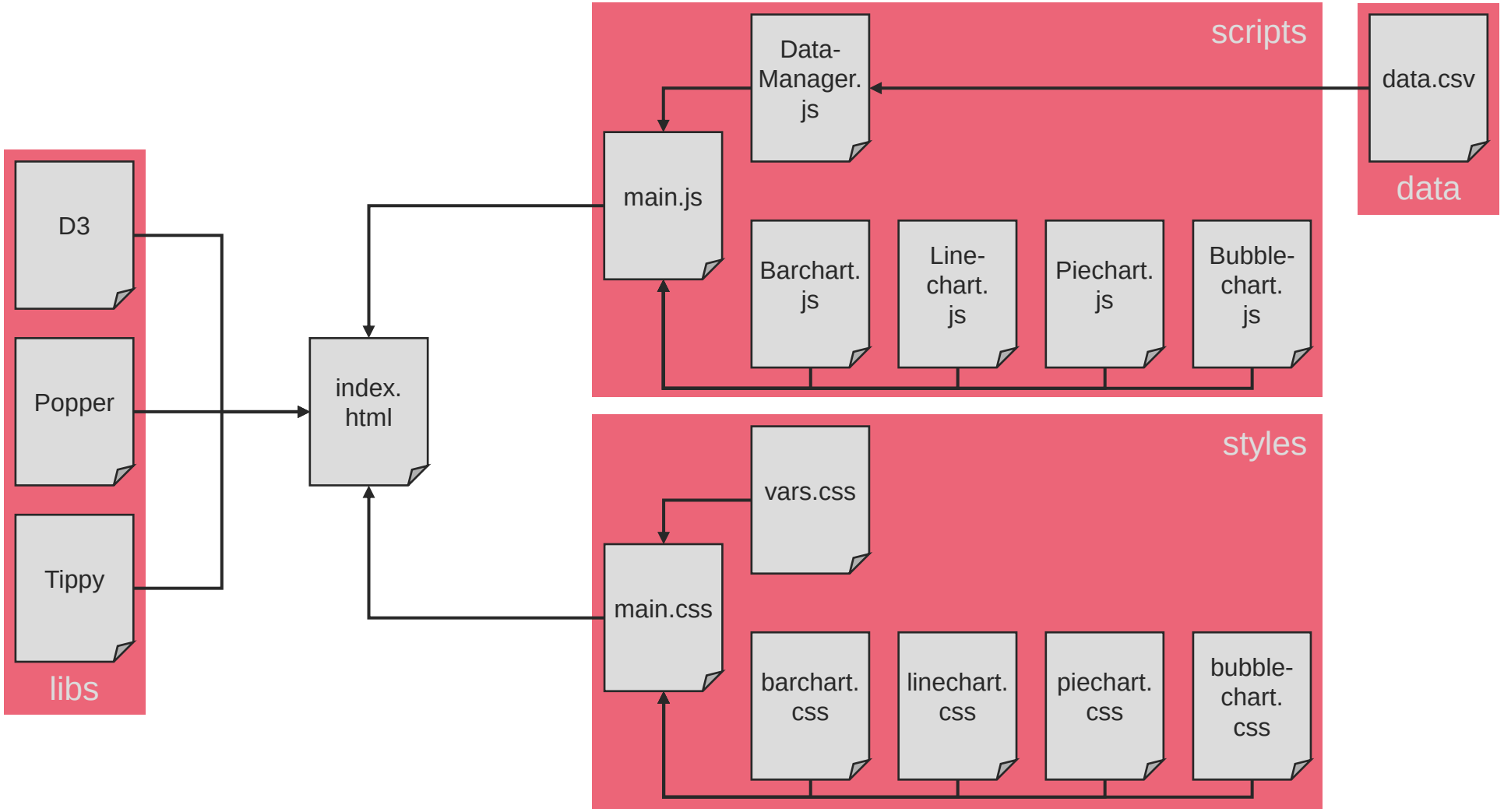


Our App Architecture



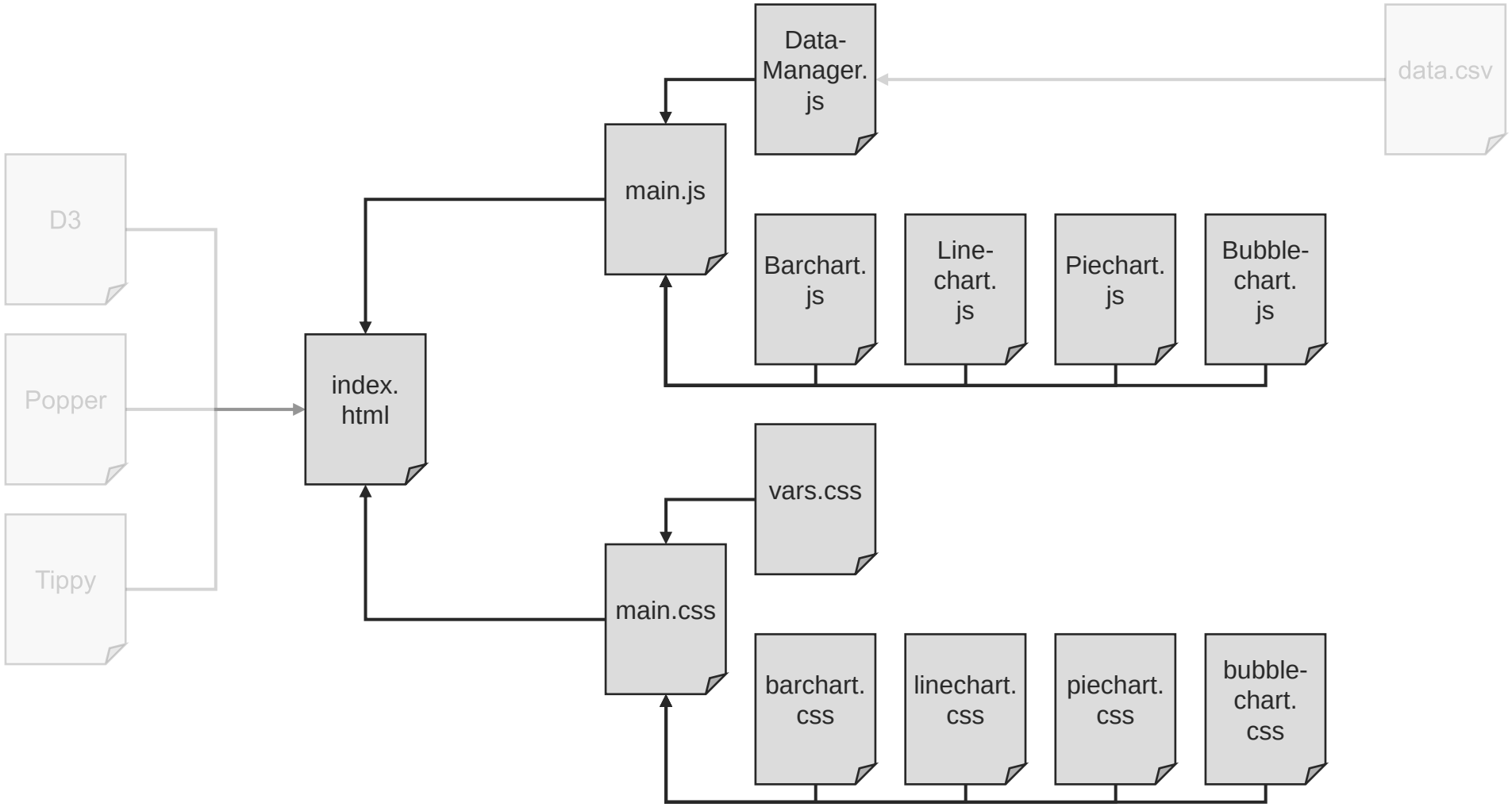
Our App Architecture

Edinburgh Napier UNIVERSITY

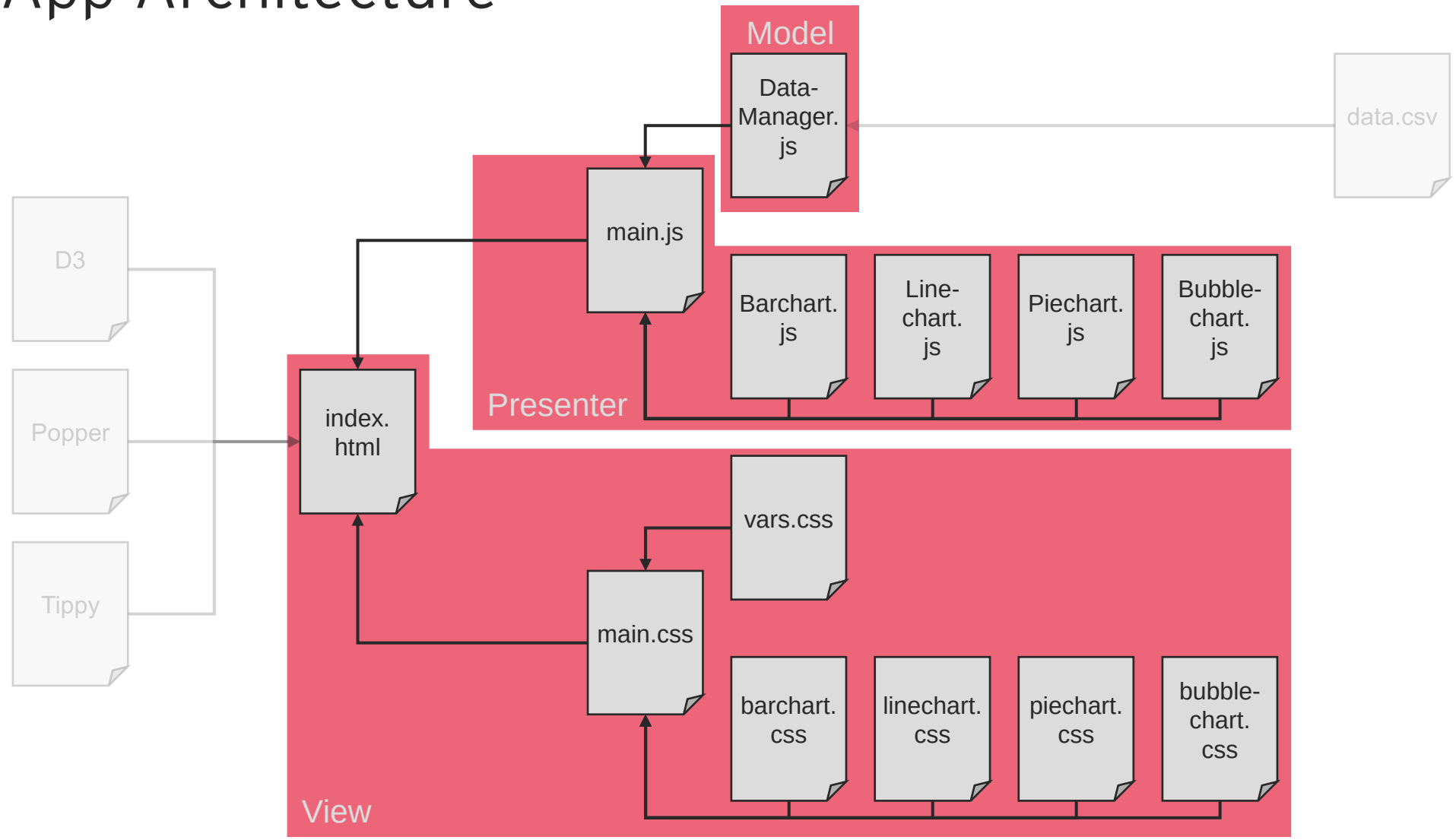


Our App Architecture

Edinburgh Napier UNIVERSITY



Our App Architecture



Our Visualisation Modules

Visualisation.js

```
// Properties (private)
    selections
    scales
    callbacks
    data

// Constructor
    setup the selections

// Private methods
    update scales
    update interaction

// Public methods
    render new dataset
    update callbacks
    update colour scale
```

Conclusion

Key Points

So far in this module, we have explored D3 modules:

- To select, bind and modify DOM elements with selections
  - Including implementing events handlers
- To map our data to the view with scales
  - And how to render these scales with axes
- To generate complex shapes
  - Including generating data for these shapes
- To interact with data
  - Creating filters and aggregations

Key Points

- We have also built an Application based on the Model-View-Presenter (MVP) design pattern

- In the next few weeks we will expand it with:
  - Advanced interaction behaviours
  - Handling temporal data
  - Visualisations considering Geographical and Hierarchical datasets
  - Dashboard design

Edinburgh Napier UNIVERSITY

Q&A

Q&A

- Any questions about the module so far?
  - Points you would like to revisit or expand on?
- Any questions about the first coursework?
- Any feedback about the module, lectures and practicals?
  - What is and isn’t working
  - Things you would like to see more of